A Massively Parallel Differential Evolution on GPU with CUDA

Amina Menacer, Lounis Mesrour, Lisa Adrar
Master 2 IMDS, Université de Haute-Alsace, Mulhouse, France

***Abstract*—Differential Evolution (DE) builds and evaluates its trial vectors independently, which makes it well suited to GPUs, but a naive mapping of one thread to one individual leaves most of the device idle. Starting from a CUDA Particle Swarm Optimization code, we implement DE/rand/1/bin on the GPU with two mappings, one thread per individual (version 1) and one block per individual with one thread per gene (version 2, following the design of cudaDEi), together with the self-adaptive jDE. On four shifted benchmark functions, three dimensions and three population sizes (1440 runs on a Tesla T4), version 2 is the fastest program in every configuration, $1.9\times$ to $58\times$ faster than a sequential C++ DE (up to $74\times$ for a single function). With Holm's correction over all tests, the GPU versions of DE/rand/1/bin are less accurate than the sequential DE only with $N = 500$; a sequential DE with the same generational replacement is not distinguishable from them in any configuration, so the loss comes from the replacement scheme, not from the GPU. jDE is more accurate than the sequential DE in 17 of 36 configurations. Finally, an independent implementation reproduces our errors but not those published for cudaDEi.**

***Index Terms*—CUDA, GPU, Differential Evolution, Massively Parallel Computing, Global Optimization**

I. Introduction

Population-based metaheuristics spend most of their time evaluating candidate solutions. In Differential Evolution (DE) [1], each of the N individuals builds a trial vector from three other individuals and evaluates it; these N tasks are independent, and inside each of them the D coordinates are processed almost independently as well. This two-level parallelism matches the hierarchy of a GPU, made of blocks of threads.

Several CUDA implementations of DE exist. cudaDE [2] assigns one block of 64 threads to one individual on a fixed grid; FPDE [3] uses a separate kernel per DE operation; cudaDEi [4] fuses mutation, crossover, evaluation and replacement into one kernel, assigns one thread to each gene and chooses the launch configuration from the problem. Qin *et al.* show that the first two designs suffer from many global-memory transfers between kernels and from low occupancy, and can be slower than a sequential DE for small problems.

This paper does not propose a new design: our version 2 follows the gene-level design of cudaDEi. Our contributions are on its evaluation:

- two GPU mappings of DE/rand/1/bin and a GPU jDE [5], built on an interface shared with a sequential reference and checked at every run;
- a campaign of 1440 runs analysed with one statistical protocol (Mann–Whitney tests with Holm's correction over all comparisons);
- a control experiment, a sequential DE with generational replacement, which locates the cause of the accuracy differences between the GPU and sequential programs;
- a reproduction attempt of the published cudaDEi errors with an independent implementation.

Section II recalls the CUDA model, Section III describes the sequential and parallel DE, Section IV reports the experiments and Section V concludes.

II. GPU Programming

CUDA programs run on a heterogeneous system: the host (CPU) executes ordinary C++ code and launches *kernels*, functions marked `__global__` that the device (GPU) executes with many threads; functions marked `__device__` are called from kernels, and `__host__ __device__` functions are compiled for both sides. A kernel is launched on a grid of blocks with the syntax `kernel<<<blocks, threads>>>`; each thread finds its position with `blockIdx` and `threadIdx`.

The GPU is made of streaming multiprocessors (SMs); the Tesla T4 used here has 40 SMs, each holding at most 1024 resident threads and 16 resident blocks. All the threads of a block run on the same SM, are executed by groups of 32 (warps), can synchronize with `__syncthreads()` and share a fast on-chip *shared memory* (48 KB per block). Threads of different blocks can only communicate through *global memory*, which is large but has a high latency; its accesses are efficient when the consecutive threads of a warp read consecutive addresses (coalesced accesses). *Constant memory* is a small cached memory whose value is broadcast when all the threads of a warp read the same address; reads of different addresses by one warp are serialized.

The *occupancy* of an SM is the ratio of resident warps to the maximum; it is limited by the block size, the registers and the shared memory used per block. High occupancy helps to hide memory latency, but does not guarantee speed. Random numbers are generated on the device with cuRAND; we use the counter-based Philox generator [6], which gives each thread an independent subsequence of one seed.

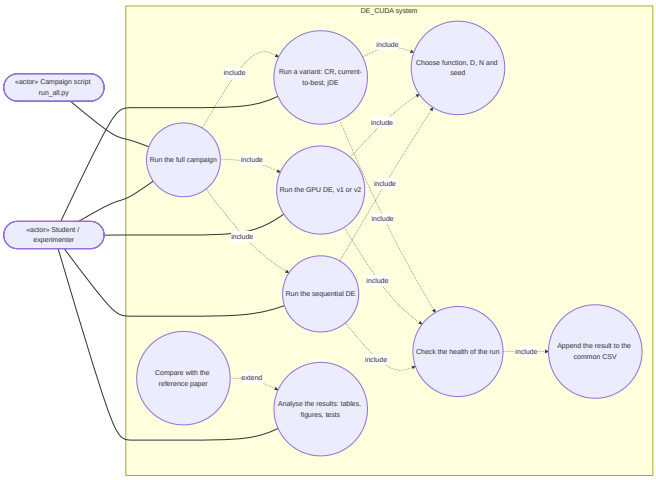


Fig. 1. UML use case diagram of the software.

III. Differential Evolution

A. Overview of the sequential DE

DE/rand/1/bin evolves a population $\{x_1, \dots, x_N\} \subset [\ell, u]^D$. At each generation, for every target x_i, three distinct indices $r_1, r_2, r_3 \neq i$ are drawn and a mutant is built:

$$v_i = x_{r_1} + F\,(x_{r_2} - x_{r_3}). \tag{1}$$

Binomial crossover mixes v_i and x_i gene by gene:

$$u_{i,j} = \begin{cases} v_{i,j} & \text{if } \mathrm{rand}_j < CR \text{ or } j = j_{\mathrm{rand}}, \\ x_{i,j} & \text{otherwise,} \end{cases} \tag{2}$$

where j_{rand} guarantees that at least one gene comes from the mutant. Genes leaving the domain are clipped to $[\ell, u]$, and u_i replaces x_i if $f(u_i) \leq f(x_i)$. We use $F = 0.5$, $CR = 0.3$ and a budget of $10^4 D$ evaluations, counted exactly (the initial population costs N evaluations and the last generation may be partial).

The sequential reference (C++, one core) replaces x_i as soon as its trial wins, so the next individuals of the same generation already see the new value (*asynchronous* DE). On the GPU, all trials of a generation are built in parallel, so the replacement takes place at the end of the generation (*generational* DE); otherwise a thread could read x_{r_1} while another one overwrites it. The sequential program also has a generational mode, used as a control in Section IV.

B. Massively parallel DE

a) Software and use cases: Fig. 1 shows the use cases. Every program (sequential, GPU, variants) takes a function, D, N and a seed, checks its own result and appends one row to a CSV file shared by all programs. The four functions, their domains and the shift vector o are written once in a header compiled for the host and the device, and o is produced by a deterministic generator (splitmix64) seeded with the function and D, so all programs optimize exactly the same problem.

b) Version 1: one thread per individual: The direct translation of the sequential code uses three kernels per generation: mutation and crossover into a trial array, evaluation of the trials, and selection. Writing the trials to their own array and selecting in a separate kernel avoids any race condition, and the population stays on the device. This mapping has two deficiencies. (i) With $N \leq 500$ and 128 threads per block, it launches at most four blocks, so at most 4 of the 40 SMs work (one SM for $N = 100$). (ii) Each thread loops over the D genes of its individual, so consecutive threads read addresses D apart and the global-memory accesses are not coalesced.

c) Version 2: one block per individual, one thread per gene: Version 2 removes both deficiencies, following cudaDEi [4]. Each generation runs two kernels:

- **kernel P**: one thread per individual draws r_1, r_2, r_3 and j_{rand} and stores them as one `int4`;
- **kernel MCER** (Mutation, Crossover, Evaluation, Replacement): block i builds u_i in shared memory, thread t handling genes $t, t+T, \dots$; each thread computes its part of $f(u_i)$, the block sums the parts by a tree reduction in $\log_2 T$ steps (Griewank reduces a sum and a product together), and the winner of u_i and x_i is written to a second population buffer.

The grid now has N blocks of T threads instead of N threads, and consecutive threads read consecutive genes. The two population buffers are swapped by exchanging pointers (double buffering). After the last generation, an argmin kernel finds the best individual, so the only transfer in the timed region is a 16-byte structure. Fig. 2 shows one run.

Two differences with cudaDEi remain. Kernels P and MCER run one after the other in one stream, whereas cudaDEi overlaps them on two streams. And the shift vector stays in constant memory as in version 1, which no longer suits version 2: thread j reads o_j, so a warp reads 32 different addresses and these reads are serialized; reading o through the read-only cache would avoid it. We did not measure the effect of either choice.

d) Block size: T is a power of two chosen automatically: $T = 32$ if $D \leq 32$ or $N \geq 4 \times \#\mathrm{SM}$, and $T = 64$ otherwise. The rule comes from a measured sweep (Section IV-C).

e) Variants: The same organization supports current-to-best/1, $v_i = x_i + F(x_{\mathrm{best}} - x_i) + F(x_{r_1} - x_{r_2})$, and jDE [5], where every individual carries its own F_i and CR_i (initially 0.5 and 0.9); with probability 0.1 each, kernel P proposes $F' \sim \mathcal{U}(0.1, 1]$ and $CR' \sim \mathcal{U}(0, 1]$, and the new values survive only if the trial wins.

f) Built-in checks: Outside the timed region, every run checks that all genes are finite and inside the domain and that the evaluation count does not exceed the budget. Version 2 and the variants also check that the GPU argmin equals a CPU minimum and that the fitness obtained by the parallel reduction equals a CPU evaluation of the same vector, within $10^{-9} \max(1, |f|)$. All the recorded runs passed these checks.

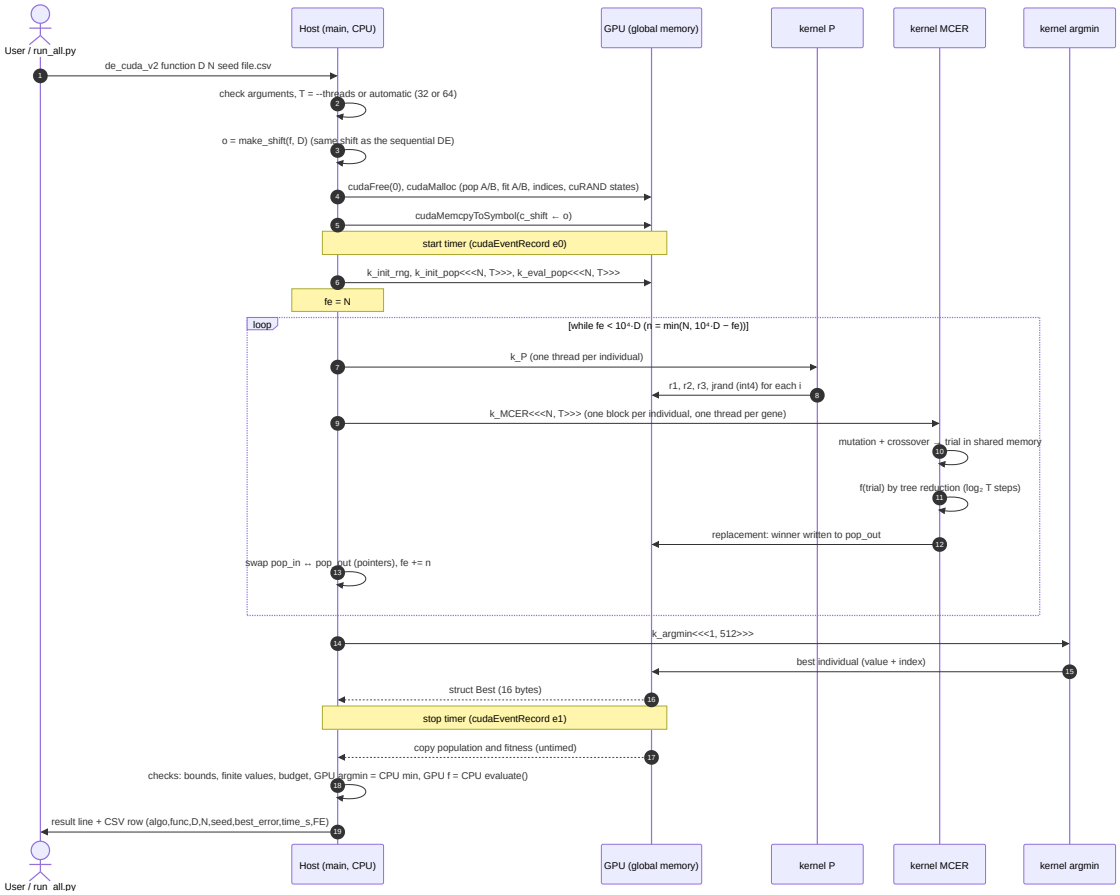


Fig. 2. UML sequence diagram of one run of version 2. In the main loop the host only launches kernels and swaps two pointers; the population never leaves the GPU.

IV. Experimental Results

A. Setup

GPU: NVIDIA Tesla T4 (Google Colab, 40 SMs), `nvcc -O3 -arch=sm_75`; CPU: Intel Xeon at 2.00 GHz, one core, `g++ -O2`, without explicit vectorization or multithreading. All computations are in double precision, which the T4 executes at 1/32 of its single-precision rate. The full campaign (CUDA 13) runs the sequential DE, versions 1 and 2 and jDE on 4 functions, $D \in \{10, 50, 100\}$ and $N \in \{50, 100, 500\}$, with seeds 1 to 10: 1440 runs. The block-size sweep (Table II) was measured earlier with CUDA 12.

Timing. The sequential time runs from the initialization of the population to the end of the main loop. The GPU time, measured with CUDA events, runs from the initialization of the random generators and of the population to the copy of the result to the host (the 16-byte best individual for version 2, the N final fitness values for version 1); it excludes the creation of the CUDA context, the allocations, the copy of o and the final copy of the population used by the checks. Over the 10 runs of a configuration, the GPU times vary little (median coefficient of variation 0.4 % for version 2), whereas the sequential times vary by 18 % (median) on the shared Colab CPU; speedups are therefore given with two significant digits.

Statistics. Each GPU algorithm is compared with the sequential DE by a two-sided Mann–Whitney test [8] on the 10 final errors, with Holm's correction [11] over the 108 tests of the campaign (3 algorithms $\times$ 36 configurations), $\alpha = 0.05$. Runs share a seed but not a random stream, so they are not paired and the signed-rank test [10] does not apply. Kruskal–Wallis tests [9] are corrected over the 36 configurations. Two identical samples give $p = 1$. A non-significant result with 10 runs means that no difference was detected, not that the algorithms are equivalent.

B. Benchmark functions

With $z = x - o$ (and $z = x - o + 1$ for Rosenbrock), following CEC 2005 [7]:

$$f_{\mathrm{Sph}}(x) = \sum_{j=1}^{D} z_j^2,$$
$$f_{\mathrm{Ros}}(x) = \sum_{j=1}^{D-1} 100\,(z_j^2 - z_{j+1})^2 + (z_j - 1)^2,$$
$$f_{\mathrm{Gri}}(x) = \sum_j \frac{z_j^2}{4000} - \prod_j \cos\frac{z_j}{\sqrt{j}} + 1,$$
$$f_{\mathrm{Ras}}(x) = \sum_j z_j^2 - 10\cos(2\pi z_j) + 10,$$

TABLE I
MEAN SEQUENTIAL TIME AND SPEEDUPS OF THE GPU PROGRAMS OVER THE SEQUENTIAL DE: RATIO OF THE MEAN TIMES POOLED OVER THE FOUR FUNCTIONS, WITH THE RANGE OVER THE FUNCTIONS IN BRACKETS (10 RUNS PER FUNCTION).

D	N	seq. (s)	v1	v2	jDE
10	50	0.033	0.81 (0.73–0.90)	1.9 (1.5–2.2)	1.6 (1.3–1.8)
10	100	0.034	1.4 (1.3–1.5)	3.8 (2.8–5.0)	3.1 (2.2–4.0)
10	500	0.039	7.9 (6.7–8.3)	13 (9.2–16)	11 (7.6–14)
50	50	0.73	1.0 (0.73–1.4)	8.1 (6.2–11)	6.5 (5.0–9.3)
50	100	0.73	1.6 (1.1–2.0)	15 (11–21)	11 (8.5–16)
50	500	0.78	7.7 (6.0–9.5)	40 (33–54)	35 (28–48)
100	50	2.8	1.0 (0.73–1.5)	13 (10–19)	11 (8.1–16)
100	100	2.9	1.6 (1.1–2.3)	24 (19–35)	18 (15–26)
100	500	3.2	7.2 (6.3–9.1)	58 (50–74)	51 (43–68)

on $[-100, 100]^D$ (Sphere, Rosenbrock), $[-600, 600]^D$ (Griewank) and $[-5, 5]^D$ (Rastrigin). The shift o is drawn uniformly in 80 % of the domain and the CEC bias is not added. Each program returns the error $f(x) - f(o)$. Its resolution differs between functions: for Griewank $(1 - \prod \cos)$ and Rastrigin $(10 - 10 \cos)$ the cancellation makes errors below about 10^{-15} unmeasurable (they are returned as 0 or as a few units of 10^{-16}), whereas for Sphere an error of 0 means $x = o$ exactly in double precision. The functions are shifted but not rotated; Sphere and Rastrigin are separable and Griewank becomes nearly separable as D grows.

C. Running time

a) Campaign: Table I and Fig. 3. Version 2 is the fastest program in all 36 configurations. Its speedup over the sequential DE grows with D and N, from $1.9\times$ ($D = 10$, $N = 50$) to $58\times$ ($D = 100$, $N = 500$) on times pooled over the four functions, and from $1.5\times$ to $74\times$ for single functions. Version 1 is slower than the sequential DE for some functions with $N = 50$ (down to $0.73\times$), stays below $2.3\times$ with $N \leq 100$, and reaches $6\times$ to $9.5\times$ with $N = 500$, when it launches four blocks. Version 2 is thus up to about $15\times$ faster than version 1 ($D = 100$, $N = 100$). Since version 1 was not tuned (blocks of 32 threads would use four SMs instead of one for $N = 100$), this ratio measures the benefit of the gene-level mapping over this baseline, not over the best one-thread-per-individual code. jDE costs 10 to 33 % more time than version 2, depending on the configuration.

These speedups are obtained at equal budget, not at equal quality: where the GPU versions are less accurate (Section IV-D), the sequential DE gets more from the same budget. With $D = 10$ the sequential DE runs in 0.03 s, less than the creation of a CUDA context, so the small speedups of this dimension only matter when the GPU is already initialized.

b) Block size: Table II gives the time of version 2 for several block sizes. For $D = 50$ and 100, blocks of 64 to 512 threads all reach 100 % theoretical occupancy, yet 128 and 256 threads are always slower than the best of 32 and 64: with $D = 50$ and $T = 256$, 80 % of the threads have no gene, and every extra thread lengthens the reduction and the synchronizations. The differences between 32 and 64

TABLE II
VERSION 2, RASTRIGIN: MEAN TIME (S, 3 RUNS) FOR T THREADS PER BLOCK; "AUTO" IS THE AUTOMATIC RULE.

D	N	32	64	128	256	auto
50	50	0.104	**0.091**	0.100	0.126	0.091
50	100	0.054	**0.050**	0.059	0.078	0.051
50	500	**0.021**	0.022	0.038	0.082	0.020
100	50	0.274	**0.219**	0.224	0.271	0.220
100	100	0.142	**0.125**	0.138	0.172	0.125
100	500	**0.060**	0.064	0.103	0.170	0.060

threads (6 to 25 %) are well above the run-to-run variation of the GPU. Blocks of 32 threads win with $N = 500$ (12.5 blocks per SM, 39 % theoretical occupancy) and blocks of 64 with $N \leq 100$ (at most 2.5 blocks per SM); a plausible explanation, not verified with a profiler, is that larger blocks are needed to keep enough warps resident when the grid is small. The automatic rule reproduces the best choice in the 12 measured cases (Rastrigin and Sphere, $D \in \{50, 100\}$, $N \in \{50, 100, 500\}$), but it was fitted on them, on one GPU; the threshold $4 \times \#\mathrm{SM} = 160$ is only known to lie between $N = 100$ and 500, and the branch $D \leq 32$ was not swept.

D. Solution quality

a) GPU versions against the sequential DE: Tables VI to VIII give the final errors. Against the sequential DE, version 1 is better in 0, not different in 31 and worse in 5 configurations; version 2 in 0, 32 and 4. All these differences occur with $N = 500$: Sphere for the three dimensions and Griewank for $D = 50$ and 100 (version 2 is not significantly worse on Griewank with $D = 100$ after correction). Kruskal–Wallis separates the three implementations of DE/rand/1/bin in exactly these 5 configurations. With $N = 500$ the budget allows only 200 to 2000 generations and DE/rand/1/bin has not converged (0.07 on Sphere with $D = 100$); the sequential error is then 1.3 to 4 times lower. With $N \leq 100$ no difference is detected.

b) Generational control: The sequential DE replaces an individual as soon as its trial wins, which propagates improvements within a generation; the GPU versions cannot. To test whether this explains the differences, we ran the sequential program in generational mode (trials read the previous population, winners go to a second one) on the 36 configurations with the same seeds, 360 runs. Table III summarizes the tests. After correction, the generational sequential DE differs from the asynchronous one in 6 configurations, the 5 above and Sphere with $D = 100$, $N = 100$ (an error of 10^{-28} against 0, without practical meaning), and it is not distinguishable from version 1 or version 2 in any configuration. The accuracy losses of the GPU versions are therefore explained by the generational replacement, not by the GPU implementation. Its times were measured on another machine and are not comparable with the campaign.

c) jDE: jDE is better than the sequential DE in 17 configurations, not different in 18 and worse in one (Sphere, $D = 100$, $N = 50$: 7×10^{-28} against 0). It also fails once

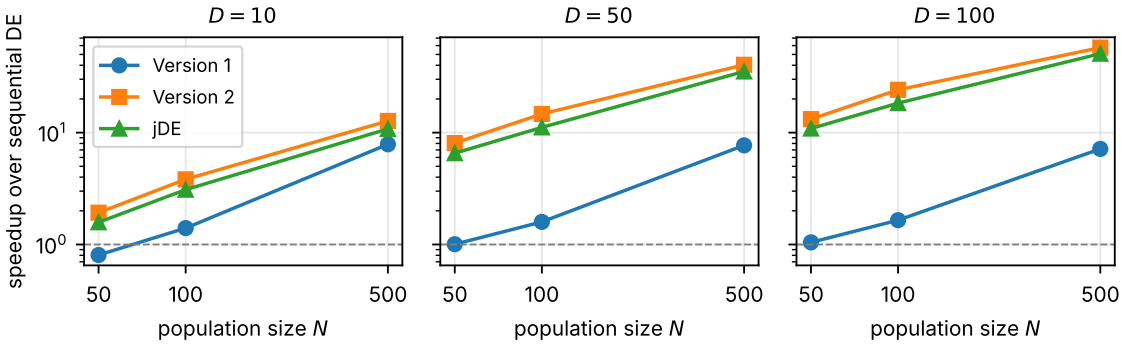


Fig. 3. Speedup over the sequential DE as a function of N (ratio of the mean times pooled over the four functions, log scales).

TABLE III
GENERATIONAL CONTROL: NUMBER OF CONFIGURATIONS (OUT OF 36) WHERE THE GENERATIONAL SEQUENTIAL DE DIFFERS FROM EACH PROGRAM (MANN–WHITNEY).

Generational DE against	raw $p < 0.05$	Holm (36 tests)
asynchronous sequential DE	9	6
GPU version 1	1	0
GPU version 2	1	0

without significance: on Griewank with $D = 100$, $N = 50$, some runs do not reach the optimum (mean 0.044 against 0). With $N \leq 100$ its gain comes mostly from Rastrigin, which it solves for $D = 50$ and 100 where DE/rand/1/bin stays near 165 and 535: on a separable function a small CR is favourable (Table IV), and jDE adapts CR towards such values. With $N = 500$ it is better in 11 of the 12 configurations (all but Sphere with $D = 10$), including the non-separable Rosenbrock (104 against 2.2×10^4 for $D = 100$): there the budget allows few generations, and the adapted F_i and CR_i converge much faster than the fixed $F = 0.5$, $CR = 0.3$. Counting as a success a final error below 10^{-8}, as Qin *et al.*, the sequential DE and both GPU versions succeed in 140 of their 360 runs and jDE in 223. These gains are algorithmic: they are not a property of the GPU, and a sequential jDE should show them as well.

d) DE variants: Table IV compares six variants of the GPU program for $D = 50$, $N = 100$ (10 runs): rand/1 with $CR \in \{0.1, 0.3, 0.5, 0.9\}$, current-to-best/1 (ctb1) and jDE. With $CR = 0.3$ the variant program reproduces version 2 bit for bit. Kruskal–Wallis detects a difference on Sphere, Rosenbrock and Rastrigin ($p < 10^{-6}$); all variants solve Griewank. On Rastrigin a small CR is best ($CR = 0.1$ reaches 10^{-11}), on the non-separable Rosenbrock a large one ($CR = 0.9$). ctb1 converges prematurely (mean 1726, median 341 on Rosenbrock) and costs about 40 % more time.

E. Comparison with cudaDEi

Qin *et al.* [4] use the same algorithm, parameters, functions and budget, with N up to 1000, 25 runs and a GTX 560. Their times are not comparable with ours: the GPUs are

TABLE IV
DE VARIANTS ON THE GPU, $D = 50$, $N = 100$: MEAN FINAL ERROR OVER 10 RUNS AND MEAN KRUSKAL–WALLIS RANK OVER THE FOUR FUNCTIONS (LOWER IS BETTER).

Variant	Sphere	Rosenbrock	Rastrigin	rank
jDE	0	32.9	0	19.2
rand/1, CR=0.1	0	47.9	2.4×10^{-11}	28.0
rand/1, CR=0.3	0	41.1	165.5	30.5
rand/1, CR=0.9	6.1×10^{-29}	27.7	227.3	30.7
rand/1, CR=0.5	5.2×10^{-30}	36.0	255.2	33.7
ctb1, CR=0.3	2.4×10^{-28}	1726	133.8	41.0

TABLE V
MEAN FINAL ERROR: OUR VERSION 2, AN INDEPENDENT NUMPY DE/RAND/1/BIN (5 SEEDS FOR SPHERE, 3 FOR RASTRIGIN) AND CUDADEI AS PUBLISHED [4].

Function	D	N	GPU v2	NumPy	cudaDEi
Sphere	10	500	9.6×10^{-6}	8.1×10^{-6}	$< 10^{-8}$
Sphere	50	500	2.4×10^{-4}	2.6×10^{-4}	$< 10^{-8}$
Sphere	100	500	0.098	0.097	$< 10^{-8}$
Rastrigin	50	100	165	147–174	2.94

seven years apart, their sequential DE runs on another CPU, and the precision and timed region are not reported. Their errors are, and Table V shows two gaps. On Sphere with $N = 500$ they report a success rate of 1 for the three dimensions, whereas our DE has not converged. On Rastrigin with $D = 50$, $N = 100$ they report 2.94 against 165. An independent NumPy implementation of DE/rand/1/bin, written from the equations of Section III, reproduces our values in both cases (for Rastrigin, whatever the bound handling), and still does not reach the published Sphere errors if the domain is reduced to $[-5.12, 5.12]^D$. The published results are therefore not reproducible from the paper's description; we could not identify the difference.

TABLE VI

FINAL ERROR FOR $N = 50$: MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER 10 RUNS. A SUPERSCRIPT MARKS A RESULT SIGNIFICANTLY BETTER ($+$) OR WORSE ($-$) THAN THE SEQUENTIAL DE (MANN–WHITNEY, HOLM'S CORRECTION OVER THE 108 TESTS OF THE CAMPAIGN). FOR GRIEWANK AND RASTRIGIN, 0 MEANS AN ERROR BELOW ABOUT 10^{-15}; FOR SPHERE, $x = o$ EXACTLY. ON ROSENBROCK WITH $D = 100$, ONE SEQUENTIAL RUN ENDS AT 137, THE NINE OTHERS BETWEEN 81 AND 87.

Function	D	Sequential DE	GPU v1	GPU v2	GPU jDE
Sphere	10	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	50	0 ± 0	0 ± 0	0 ± 0	$5.0{\times}10^{-30} \pm 1.6{\times}10^{-29}$
	100	0 ± 0	0 ± 0	0 ± 0	$6.8{\times}10^{-28} \pm 7.2{\times}10^{-28-}$
Rosenbrock	10	2.03 ± 1.71	0.721 ± 1.11	0.463 ± 0.537	0.399 ± 1.26
	50	37.9 ± 1.58	39.2 ± 1.33	39.5 ± 1.36	$1.46 \pm 2.41^{+}$
	100	89.6 ± 16.9	84.6 ± 2.28	85.1 ± 2.95	79.4 ± 64.3
Griewank	10	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	50	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	100	0 ± 0	0 ± 0	0 ± 0	0.0440 ± 0.0555
Rastrigin	10	0 ± 0	0 ± 0	0 ± 0	0.0995 ± 0.315
	50	151 ± 6.91	146 ± 6.92	139 ± 7.82	$0 \pm 0^{+}$
	100	493 ± 10.2	495 ± 18.4	495 ± 11.4	$0.497 \pm 0.846^{+}$

TABLE VII

FINAL ERROR FOR $N = 100$ (SAME CONVENTIONS AS TABLE VI).

Function	D	Sequential DE	GPU v1	GPU v2	GPU jDE
Sphere	10	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	50	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	100	0 ± 0	$1.9{\times}10^{-30} \pm 4.4{\times}10^{-30}$	$5.6{\times}10^{-29} \pm 1.0{\times}10^{-28}$	$2.1{\times}10^{-29} \pm 6.4{\times}10^{-29}$
Rosenbrock	10	3.04 ± 1.87	3.31 ± 1.89	3.09 ± 2.28	$0.0503 \pm 0.118^{+}$
	50	41.0 ± 0.353	42.8 ± 5.35	41.1 ± 0.493	32.9 ± 21.6
	100	89.6 ± 0.201	90.0 ± 0.694	89.7 ± 0.195	112 ± 31.7
Griewank	10	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	50	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	100	0 ± 0	0 ± 0	0 ± 0	0 ± 0
Rastrigin	10	0 ± 0	0 ± 0	0 ± 0	0 ± 0
	50	168 ± 6.01	165 ± 9.99	165 ± 10.3	$0 \pm 0^{+}$
	100	542 ± 10.9	532 ± 14.5	533 ± 14.4	$0 \pm 0^{+}$

TABLE VIII

FINAL ERROR FOR $N = 500$ (SAME CONVENTIONS AS TABLE VI).

Function	D	Sequential DE	GPU v1	GPU v2	GPU jDE
Sphere	10	$2.4{\times}10^{-6} \pm 3.1{\times}10^{-7}$	$7.8{\times}10^{-6} \pm 2.9{\times}10^{-6-}$	$9.6{\times}10^{-6} \pm 3.1{\times}10^{-6-}$	$1.6{\times}10^{-6} \pm 7.3{\times}10^{-7}$
	50	$1.1{\times}10^{-4} \pm 1.1{\times}10^{-5}$	$2.3{\times}10^{-4} \pm 2.8{\times}10^{-5-}$	$2.4{\times}10^{-4} \pm 2.8{\times}10^{-5-}$	$7.3{\times}10^{-11} \pm 1.8{\times}10^{-11+}$
	100	$0.0733 \pm 5.5{\times}10^{-3}$	$0.0979 \pm 0.0117^{-}$	$0.0982 \pm 6.5{\times}10^{-3-}$	$4.3{\times}10^{-15} \pm 1.3{\times}10^{-15+}$
Rosenbrock	10	53.3 ± 15.9	48.0 ± 15.6	47.5 ± 16.1	$19.7 \pm 8.52^{+}$
	50	358 ± 53.2	434 ± 55.3	400 ± 33.6	$45.2 \pm 1.01^{+}$
	100	$2.2{\times}10^{4} \pm 3.4{\times}10^{3}$	$2.3{\times}10^{4} \pm 3.4{\times}10^{3}$	$2.5{\times}10^{4} \pm 3.3{\times}10^{3}$	$104 \pm 24.5^{+}$
Griewank	10	0.143 ± 0.0245	0.147 ± 0.0304	0.145 ± 0.0335	$0.0755 \pm 0.0208^{+}$
	50	$2.1{\times}10^{-4} \pm 3.0{\times}10^{-5}$	$4.1{\times}10^{-4} \pm 5.9{\times}10^{-5-}$	$4.3{\times}10^{-4} \pm 4.3{\times}10^{-5-}$	$9.9{\times}10^{-11} \pm 2.0{\times}10^{-11+}$
	100	$0.0439 \pm 3.3{\times}10^{-3}$	$0.0557 \pm 5.0{\times}10^{-3-}$	$0.0571 \pm 5.4{\times}10^{-3}$	$1.9{\times}10^{-15} \pm 9.2{\times}10^{-16+}$
Rastrigin	10	4.48 ± 0.915	4.66 ± 0.789	4.92 ± 1.18	$2.19 \pm 0.564^{+}$
	50	224 ± 6.49	229 ± 7.92	222 ± 7.61	$17.9 \pm 2.67^{+}$
	100	628 ± 14.2	623 ± 17.5	623 ± 17.8	$35.4 \pm 3.22^{+}$

V. Conclusion

Mapping one individual to one thread gives a correct GPU DE but not a fast one: with up to 500 individuals, most of the 40 SMs of a T4 stay idle. Mapping one individual to one block and one gene to one thread, fusing the four DE operators and keeping the population and the search for the best individual on the device makes the GPU program faster than the sequential DE in every configuration, by $1.9\times$ to $58\times$ on pooled times. The GPU versions lose accuracy only with $N = 500$, and a generational sequential DE loses the same accuracy, which locates the cause in the replacement scheme. The self-adaptive jDE, 10 to 33 % slower than version 2, is the most accurate program. Measured timings, not theoretical occupancy, determined the block size.

Limitations. The speedups come from one GPU, in double precision, against a single-core sequential code compiled with `-O2`, and are at equal budget, not at equal quality. Version 1 was not tuned. No profiler was used, so the explanations based on occupancy and memory accesses are consistent with the timings but not measured. Quality was compared on 10 runs per configuration, on shifted but unrotated functions. The block-size rule was fitted on 12 cases. The cudaDEi errors could not be reproduced.

Future work. Overlapping kernels P and MCER on two streams; reading o through the read-only cache; warp-level reductions (`__shfl_down_sync`) and several individuals per block when D is small; profiling with Nsight Compute; a sequential jDE and effect sizes; rotated and composite functions and modern adaptive DE (SHADE, L-SHADE).

The code, the raw results of the 1440 runs and of the control, and the analysis scripts are available at https://github.com/amenacer/DE_CUDA.

Acknowledgment

This work was carried out in the Massively Parallel Computing course of the Master 2 IMDS at Université de Haute-Alsace, from a CUDA PSO code provided by the course.

References

[1] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.

[2] L. de P. Veronese and R. A. Krohling, "Differential evolution algorithm on the GPU with C-CUDA," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, Barcelona, Spain, 2010.

[3] S. J. Domínguez González and N. G. Barriga, "Fully parallel differential evolution," in *Competition on GPUs for Genetic and Evolutionary Computation, GECCO*, Dublin, Ireland, 2011.

[4] A. K. Qin, F. Raimondo, F. Forbes, and Y. S. Ong, "An improved CUDA-based implementation of differential evolution on GPU," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*, Philadelphia, USA, 2012.

[5] J. Brest, S. Greiner, B. Bošković, M. Mernik, and V. Žumer, "Self-adapting control parameters in differential evolution: a comparative study on numerical benchmark problems," *IEEE Trans. Evol. Comput.*, vol. 10, no. 6, pp. 646–657, 2006.

[6] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel random numbers: as easy as 1, 2, 3," in *Proc. SC11*, 2011, Art. 16.

[7] P. N. Suganthan *et al.*, "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Nanyang Technological Univ., Singapore, and KanGAL Rep. 2005005, 2005.

[8] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 1947.

[9] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *J. Amer. Statist. Assoc.*, vol. 47, no. 260, pp. 583–621, 1952.

[10] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[11] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.